



JDK中的对象集合及相关接口

北京理工大学计算机学院
金旭亮

何谓“集合”？

“**集合 (collection)**”这个概念，源自于数学。特别地，在数学中，将放在集合中的数称为“**元素 (element)**”。

计算机科学中的集合，其实就是从数学中的“集合”概念衍生出来的，它可以看成是一组对象的“容器”，你可以把若干个对象“塞”进集合这一“容器”中，不同的集合，以不同的方式管理（或者说“保存”）这些对象，相应地，外界也需要使用相应的方法从集合中取出这些对象。



Java集合的特点



JDK中包容相当多的集合，JDK的设计者将各种集合的共性和特性进行了仔细地分析，然后，将这些共性和特性抽象为接口，并且在接口之间建立了继承关系。



特定类型的集合，实现特定的接口。



“接口与实现分离”，是JDK集合设计与实现的基本原则。

JDK中的主要集合类型

JDK中的对象集合，依据其实现的接口，大致上可以划分为以下四类，各有各的特性：

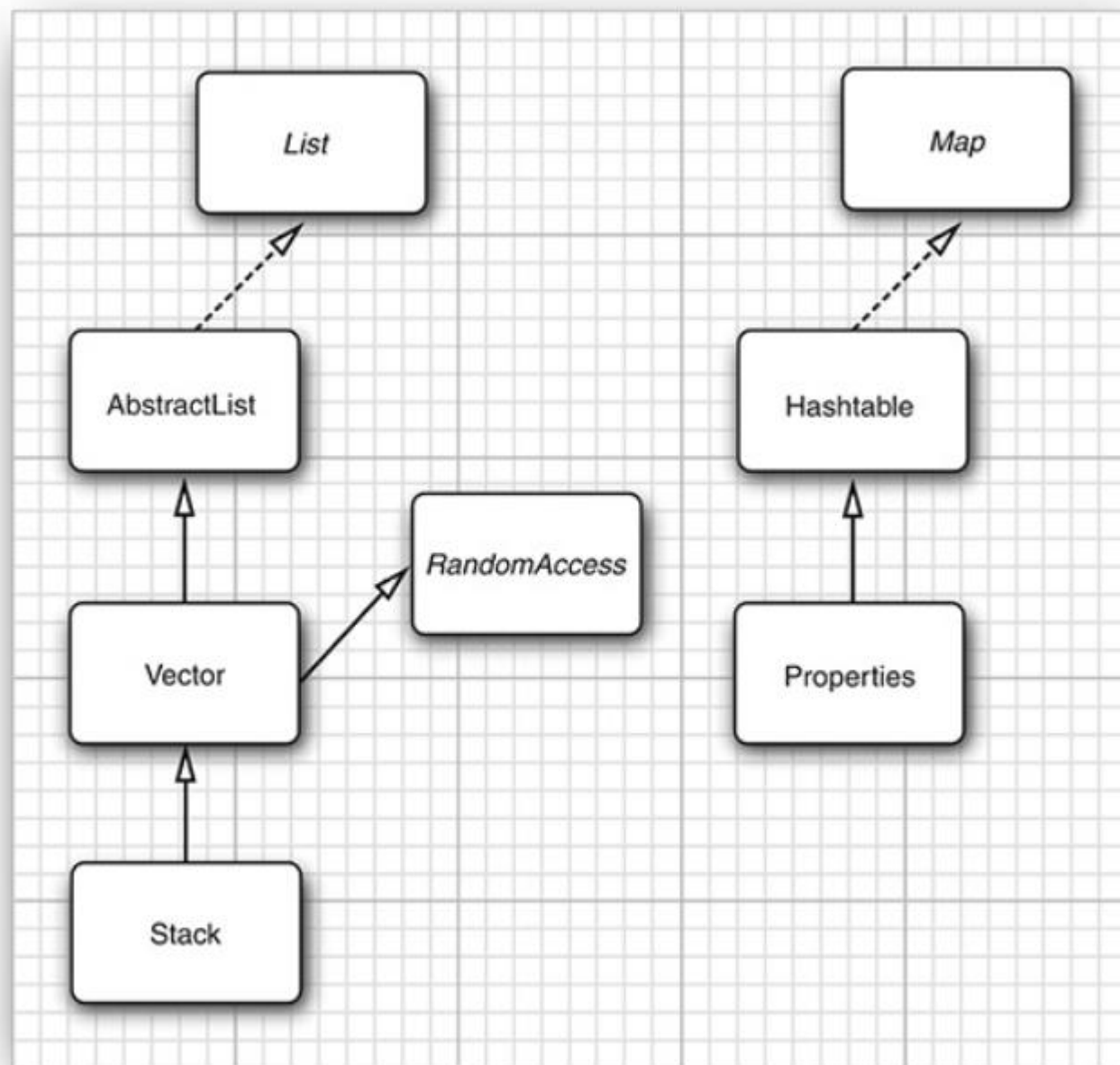
1. **Set**: 无序、不可重复的集合
2. **List**: 有序、可重复的集合
3. **Map**: 具有映射关系的集合
4. **Queue**: 具有队列特性的集合



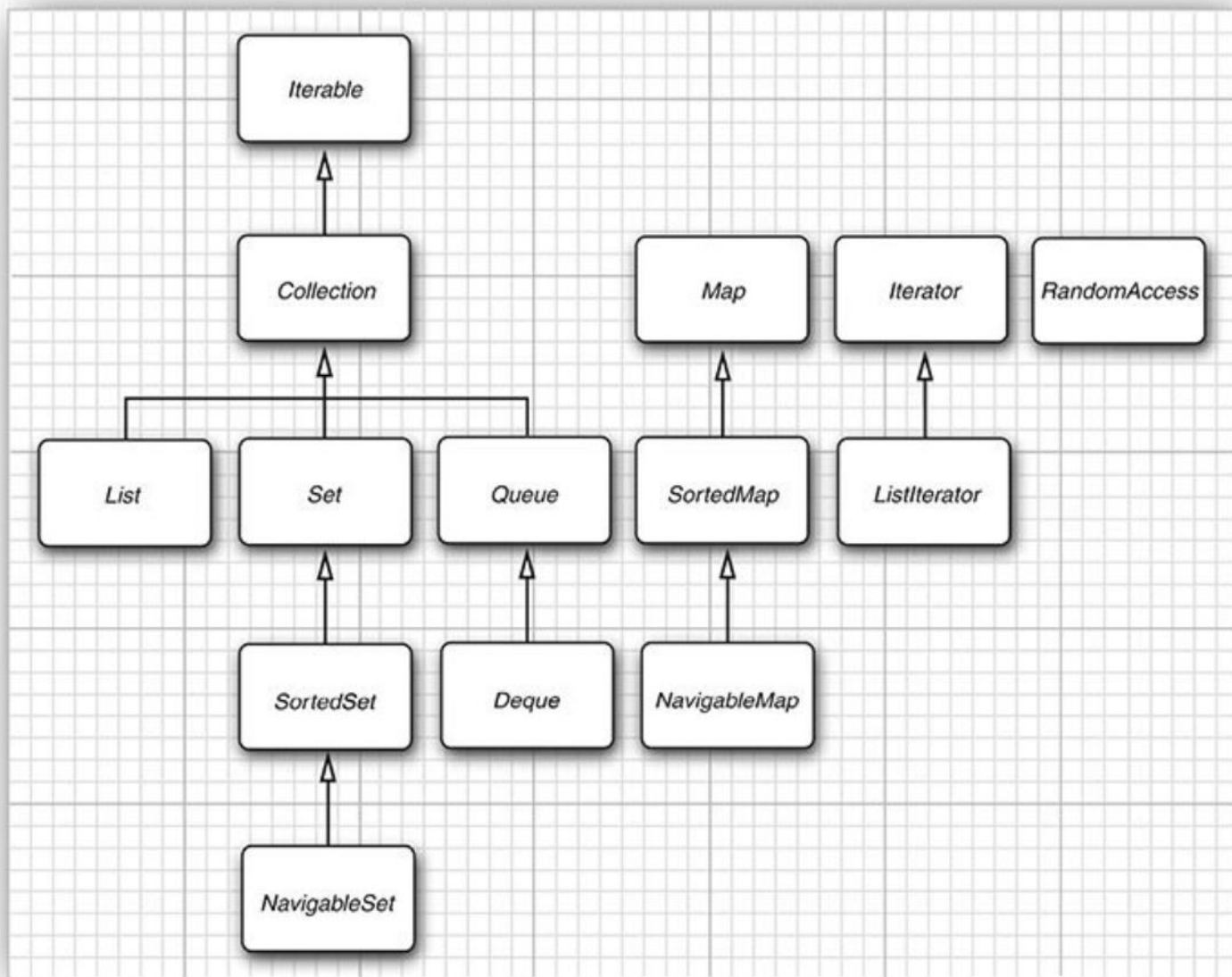
早期的集合类

早在JDK 1.0中就内置了集合类型（如右图所示）。在泛型技术引入之后，这些集合接口或类型大都被转换为了相应的泛型版本，有些则被一些新的集合类型所替换。

在实际开发中，推荐使用集合类型的泛型版本，并且在功能一致的前提下，尽量使用新的而不是老的集合类型。



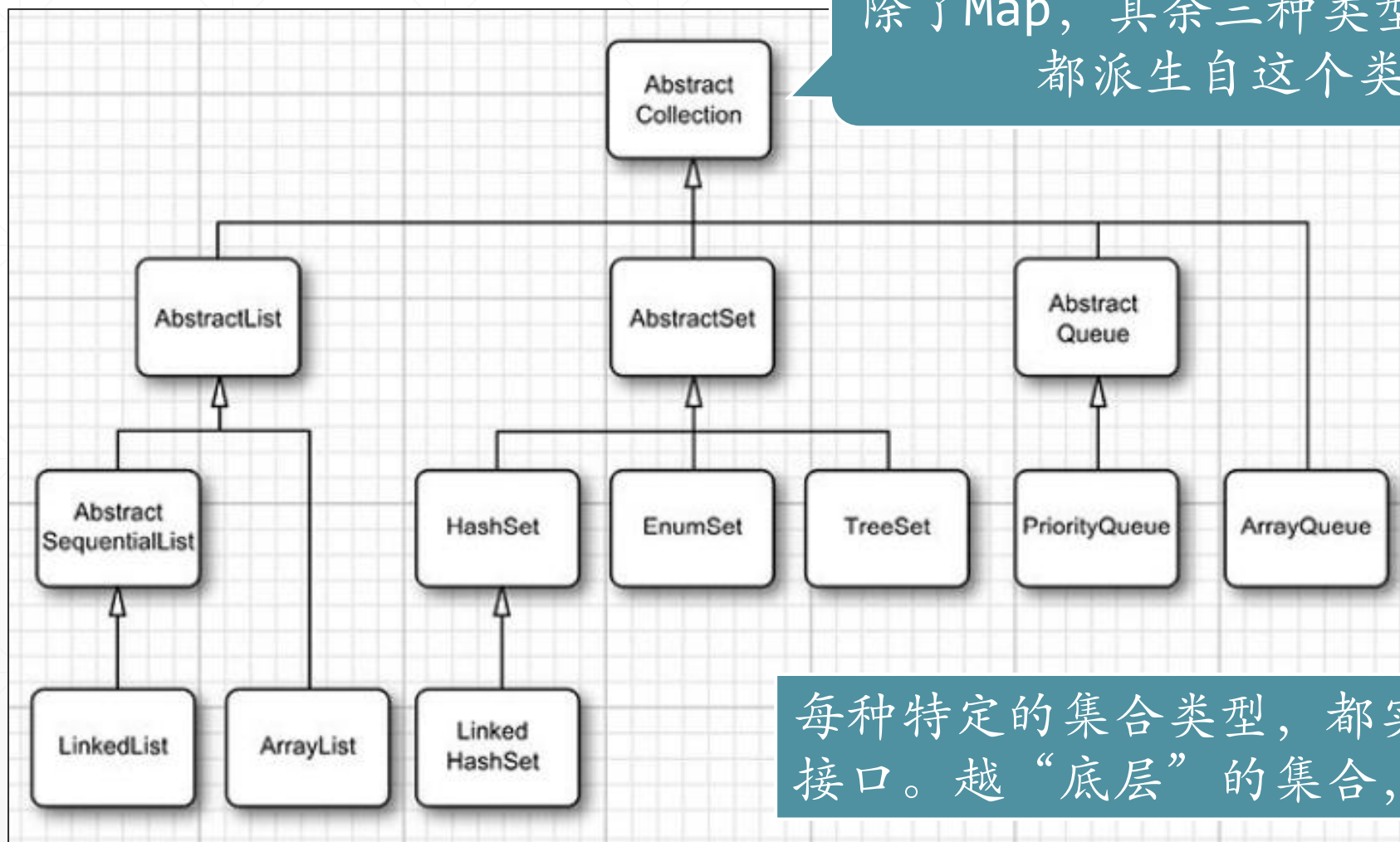
JDK当前版本中的主要集合接口类型



为了简洁，图中的各个类型都省去了泛型参数。

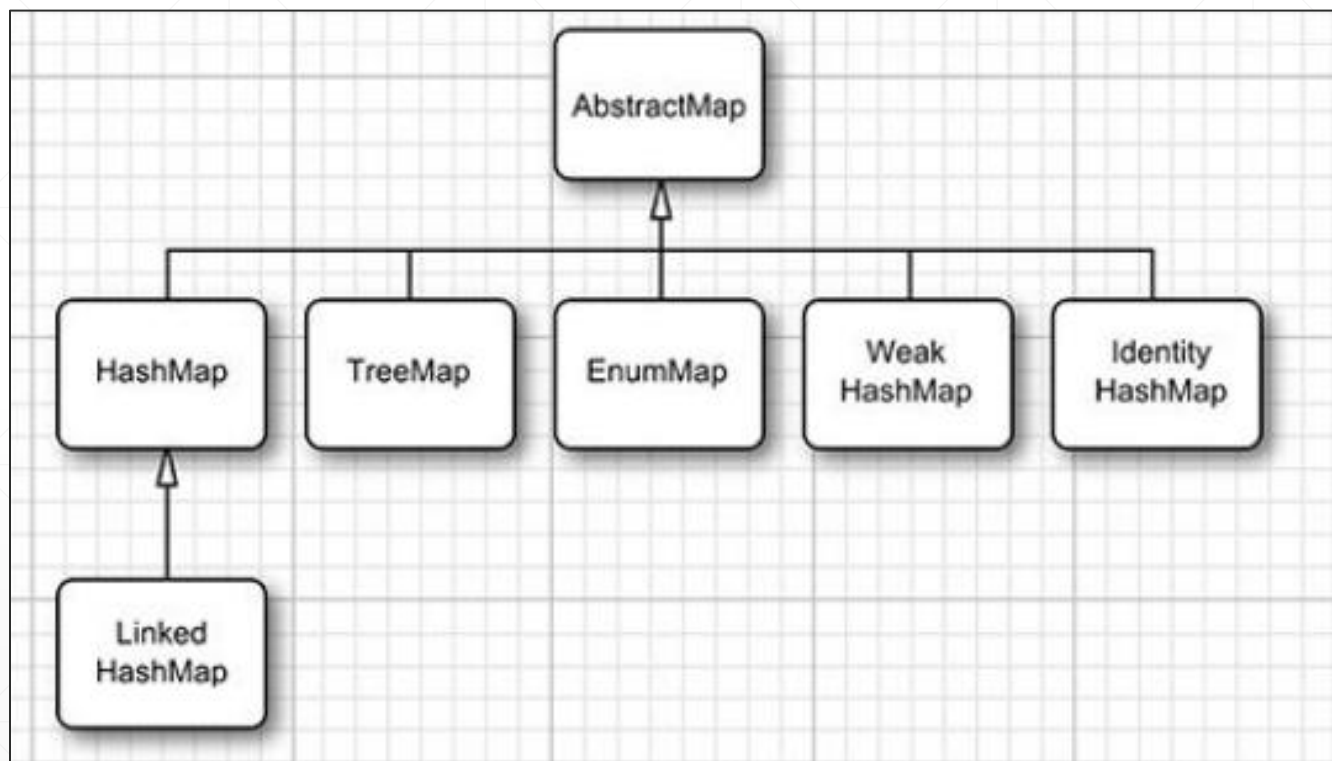
JDK中的主要集合类型-1

除了Map，其余三种类型的集合，都派生自这个类。



每种特定的集合类型，都实现了相应的集合接口。越“底层”的集合，其功能越丰富。

JDK中的主要集合类型-2



由于要保存的数据比较特殊，不是“一个”而是“一对”，所以，Map自成体系，所有的Map，都派生自AbstractMap这个抽象类。

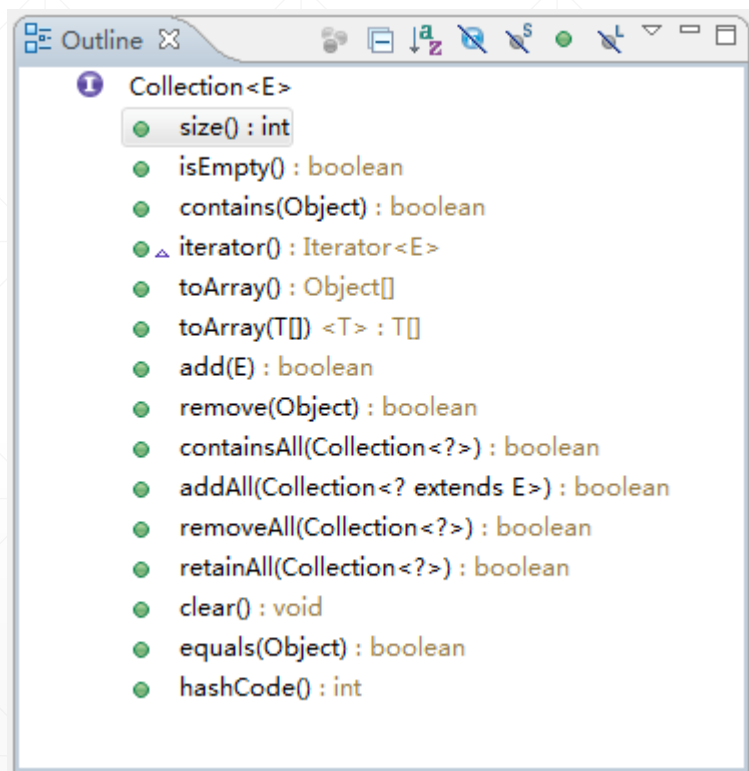
JDK中各集合类的设计思路

- JDK中的集合类型，在底层分别使用了不同的“基础集合类型”，再加上其他代码，构造出功能增强的新集合类型，这实际上就是一种“对象组合”的编程方式。
- 这些基础的集合类型包括：Array（数组），Linked List（链表），Hash Table（哈希表）和Tree（树）

外层集合类型

基础集合类型

Collection接口



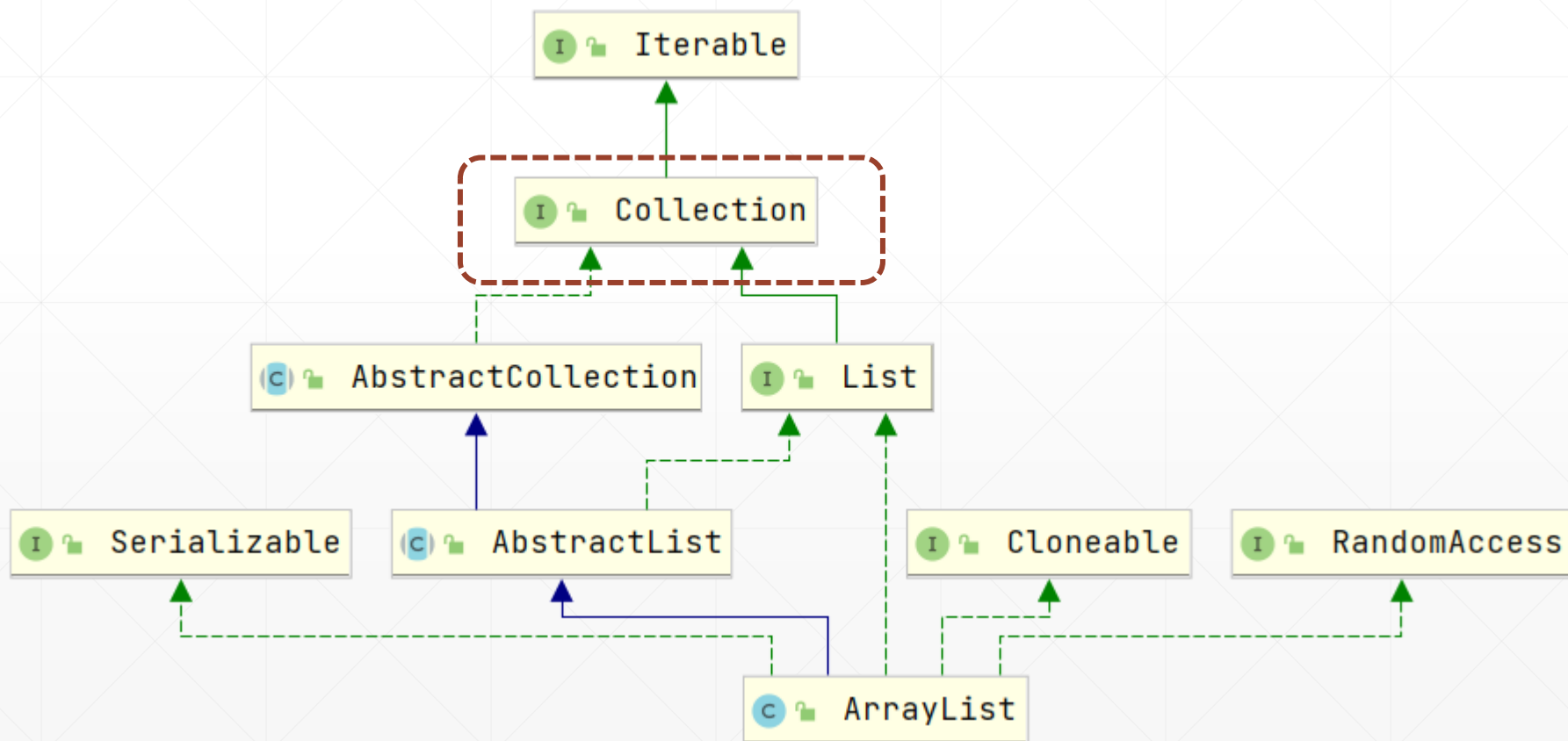
Collection是JDK中集合类型的最上层接口，很多相关接口和集合类都派生自（或实现）它。

Collection接口定义了一个对象集合应该提供的最基本的功能



JDK中的ArrayList

ArrayList实现了诸多的接口，其中就有Collection接口：



通过示例学用ArrayList

首先，我们创建一个ArrayList对象：

```
//ArrayList实现了Collection接口  
Collection arrayList = new ArrayList();
```

TestCollection.java



出于演示目的，这里没有采用泛型集合类型，而是使用了JDK中最原始的集合类型。在实际开发中，现在应该使用集合类的泛型版本，本例仅供展示和学习使用。

向集合中添加和移除元素

```
private static void addAndRemoveElements(Collection arrayList) {  
    //添加元素  
    arrayList.add("张三");  
    //虽然集合里不能放基本类型的值, 但Java支持自动装箱  
    arrayList.add(6);  
    System.out.println("ArrayList包容的所有元素: " + arrayList);  
    System.out.println("ArrayList集合的元素个数为:"  
        + arrayList.size());  
    //删除指定元素 (注意, 这里的参数不是ArrayList元素下标, 而是元素本身)  
    var result :boolean = arrayList.remove(0: 6);  
    if (result) {  
        System.out.println("删除元素6成功! 当前ArrayList集合的元素个数为:"  
            + arrayList.size());  
        System.out.println("当前ArrayList包容的内容: " + arrayList);  
    } else {  
        System.out.println("删除失败");  
    }  
}
```

- add: 添加元素
- remove: 移除元素
- size: 集合中的元素个数
- toString: 查看集合内容

Run: TestCollection ×

```
"C:\Program Files\Java\jdk-15\bin\java.exe"  
ArrayList包容的所有元素: [张三, 6]  
ArrayList集合的元素个数为:2  
删除元素6成功! 当前ArrayList集合的元素个数为:1  
当前ArrayList包容的内容: [张三]  
  
Process finished with exit code 0
```

查看集合内容的便捷方法

```
System.out.println(集合变量);
```

当使用 `System.out.println()` 方法来输出集合对象时，将输出 `[ 元素一, 元素二, ... ]` 的形式，这是因为所有的 `Collection` 实现类都重写了 `toString()` 方法，该方法可以一次性地输出集合中的所有元素。



注意!

JDK 中的对象集合有一个重要的限制:

Java 集合中不能保存原始数据类型 (比如 int, float) 的数据

请看以下代码:

```
Collection collection = new ArrayList();  
collection.add(2);
```



自动装箱

将会导致 `Integer.valueOf()` 方法被调用, 最终在集合里保存的是 `Integer` 对象引用而不是整数值。

在集合中查询特定的元素

使用contains()方法在集合中查找特定的元素，这一方法在内部调用对象的equals()进行判等比对：

```
//判断是否包含指定字符串  
System.out.println("ArrayList集合是否包含“张三”字符串? " +  
    arrayList.contains("张三"));
```



```
ArrayList集合是否包含“张三”字符串? true
```

不同类型集合协同工作

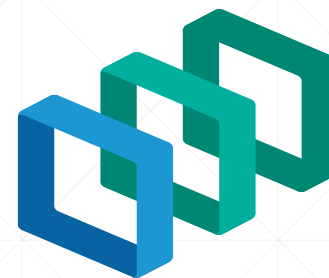
```
private static void arrayListAndHashSet(Collection arrayList) {  
    //HashSet也同样实现了Collection接口  
    Collection<String> books = new HashSet<String>();  
    books.add("book1");  
    books.add("book2");  
    arrayList.add("book1");  
    System.out.println("ArrayList集合的元素: " + arrayList);  
  
    //集合之间可以相互配合工作, 因为它们实现了相同的集合接口  
    System.out.println("ArrayList集合是否完全包含books集合? " +  
        arrayList.containsAll(books));  
    //用ArrayList集合减去在books集合里的元素  
    arrayList.removeAll(books);  
    System.out.println("ArrayList集合的元素: " + arrayList);  
}
```

由于JDK中的集合普遍都实现了Collection等通用接口, 所以, 基于面向对象的“多态特性”, 多种类型的集合之间可以协同工作。

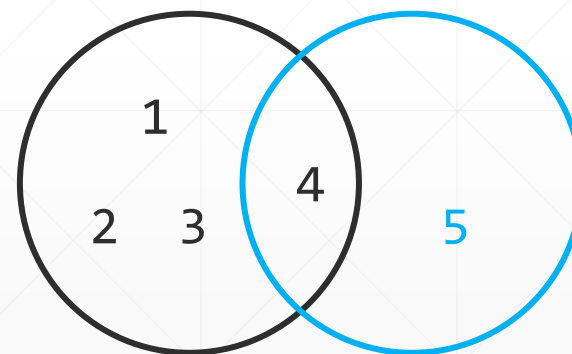
运行结果:

```
ArrayList集合的元素: [张三, book1]  
ArrayList集合是否完全包含books集合? false  
ArrayList集合的元素: [张三]
```


理解retainAll()方法



```
static void testRetainAll() {  
    var list1 = new ArrayList<Integer>();  
    list1.add(1);  
    list1.add(2);  
    list1.add(3);  
    list1.add(4);  
    var list2 = new ArrayList<Integer>();  
    list2.add(4);  
    list2.add(5);  
    //只保留list1中那些在List2中出现的元素  
    list1.retainAll(list2);  
    //输出: 4  
    System.out.println(list1);  
}
```



多学一点

集合转数组，可以使用`Collection.toArray()`方法：

```
static void collectionToArray(){  
    var stringList : List<String> = List.of("one", "two", "three");  
    var stringArr : String[] = stringList.toArray(String[]::new);  
    //输出: [one, two, three]  
    System.out.println(Arrays.toString(stringArr));  
}
```

使用`Arrays`工具类的`toString()`方法，
可以很方便地将数组中的所有元素连接
为一个字符串。

小结



本讲介绍了JDK中集合的基础知识，指出这些集合都实现了特定的接口，利用面向对象的多态特性，可以很方便地让这些集合协同工作，或者编写相应的处理集合的代码。



本讲通过一个示例展示了集合的一些基本特性，在后面的课程中，将选择那些在开发中最常见的集合，对它们进行分类介绍。